

NIST and should be deprecated in favor of Time based One-time Passwords (TOTPs), a cryptographic mechanism, or similar. NIST SP 800-63B § 5.1.3.3 recommends addressing the risks of device swap, SIM change, number porting, or other abnormal behavior, if telephone or SMS out-of-band authentication absolutely has to be supported. While this ASVS section does not mandate this as a requirement, not taking these precautions for a sensitive L2 app or an L3 app should be seen as a significant red flag.

Note that NIST has also recently provided guidance which discourages the use of push notifications. While this ASVS section does not do so, it is important to be aware of the risks of “push bombing”.

#	Description	Level
6.6.1	Verify that authentication mechanisms using the Public Switched Telephone Network (PSTN) to deliver One-time Passwords (OTPs) via phone or SMS are offered only when the phone number has previously been validated, alternate stronger methods (such as Time based One-time Passwords) are also offered, and the service provides information on their security risks to users. For L3 applications, phone and SMS must not be available as options.	2
6.6.2	Verify that out-of-band authentication requests, codes, or tokens are bound to the original authentication request for which they were generated and are not usable for a previous or subsequent one.	2
6.6.3	Verify that a code based out-of-band authentication mechanism is protected against brute force attacks by using rate limiting. Consider also using a code with at least 64 bits of entropy.	2
6.6.4	Verify that, where push notifications are used for multi-factor authentication, rate limiting is used to prevent push bombing attacks. Number matching may also mitigate this risk.	3

V6.7 Cryptographic authentication mechanism

Cryptographic authentication mechanisms include smart cards or FIDO keys, where the user has to plug in or pair the cryptographic device to the computer to complete authentication. The authentication server will send a challenge nonce to the cryptographic device or software, and the device or software calculates a response based upon a securely stored cryptographic key. The requirements in this section provide implementation-specific guidance for these mechanisms, with guidance on cryptographic algorithms being covered in the “Cryptography” chapter.

Where shared or secret keys are used for cryptographic authentication, these should be stored using the same mechanisms as other system secrets, as documented in the “Secret Management” section in the “Configuration” chapter.

The requirements in this section mostly relate to § 5.1.7.2 of NIST’s Guidance.

#	Description	Level
6.7.1	Verify that the certificates used to verify cryptographic authentication assertions are stored in a way protects them from modification.	3
6.7.2	Verify that the challenge nonce is at least 64 bits in length, and statistically unique or unique over the lifetime of the cryptographic device.	3

V6.8 Authentication with an Identity Provider

Identity Providers (IdPs) provide federated identity for users. Users will often have more than one identity with multiple IdPs, such as an enterprise identity using Azure AD, Okta, Ping Identity, or Google, or consumer identity using Facebook, Twitter, Google, or WeChat, to name just a few common alternatives. This list is not an endorsement of these companies or services, but simply an encouragement for developers to consider the reality that many users have many established identities. Organizations should consider integrating with existing user identities, as per the risk profile of the IdP's strength of identity proofing. For example, it is unlikely a government organization would accept a social media identity as a login for sensitive systems, as it is easy to create fake or throw-away identities, whereas a mobile game company may well need to integrate with major social media platforms to grow their active player base.

Secure use of external identity providers requires careful configuration and verification to prevent identity spoofing or forged assertions. This section provides requirements to address these risks.

#	Description	Level
6.8.1	Verify that, if the application supports multiple identity providers (IdPs), the user's identity cannot be spoofed via another supported identity provider (eg. by using the same user identifier). The standard mitigation would be for the application to register and identify the user using a combination of the IdP ID (serving as a namespace) and the user's ID in the IdP.	2
6.8.2	Verify that the presence and integrity of digital signatures on authentication assertions (for example on JWTs or SAML assertions) are always validated, rejecting any assertions that are unsigned or have invalid signatures.	2
6.8.3	Verify that SAML assertions are uniquely processed and used only once within the validity period to prevent replay attacks.	2

#	Description	Level
6.8.4	Verify that, if an application uses a separate Identity Provider (IdP) and expects specific authentication strength, methods, or recentness for specific functions, the application verifies this using the information returned by the IdP. For example, if OIDC is used, this might be achieved by validating ID Token claims such as 'acr', 'amr', and 'auth_time'(if present). If the IdP does not provide this information, the application must have a documented fallback approach that assumes that the minimum strength authentication mechanism was used (for example, single-factor authentication using username and password).	2

References

For more information, see also:

- NIST SP 800-63 - Digital Identity Guidelines
- NIST SP 800-63B - Authentication and Lifecycle Management
- NIST SP 800-63 FAQ
- OWASP Web Security Testing Guide: Testing for Authentication
- OWASP Password Storage Cheat Sheet
- OWASP Forgot Password Cheat Sheet
- OWASP Choosing and Using Security Questions Cheat Sheet
- CISA Guidance on “Number Matching”
- Details on the FIDO Alliance

V7 Session Management

Control Objective

Session management mechanisms allow applications to correlate user and device interactions over time, even when using stateless communication protocols (such as HTTP). Modern applications may use multiple session tokens with distinct characteristics and purposes. A secure session management system is one that prevents attackers from obtaining, utilizing, or otherwise abusing a victim's session. Applications maintaining sessions must ensure that the following high-level session management requirements are met:

- Sessions are unique to each individual and cannot be guessed or shared.
- Sessions are invalidated when no longer required and are timed out during periods of inactivity.

Many of the requirements in this chapter relate to selected NIST SP 800-63 Digital Identity Guidelines controls, focusing on common threats and commonly exploited authentication weaknesses.

Note that requirements for specific implementation details of certain session management mechanisms can be found elsewhere:

- HTTP Cookies are a common mechanism for securing session tokens. Specific security requirements for cookies can be found in the “Web Frontend Security” chapter.
- Self-contained tokens are frequently used as a way of maintaining sessions. Specific security requirements can be found in the “Self-contained Tokens” chapter.

V7.1 Session Management Documentation

There is no single pattern that suits all applications. Therefore, it is not feasible to define universal boundaries and limits that suit all cases. A risk analysis with documented security decisions related to session handling must be conducted as a prerequisite to implementation and testing. This ensures that the session management system is tailored to the specific requirements of the application.

Regardless of whether a stateful or “stateless” session mechanism is chosen, the analysis must be complete and documented to demonstrate that the selected solution is capable of satisfying all relevant security requirements. Interaction with any Single Sign-on (SSO) mechanisms in use should also be considered.

#	Description	Level
7.1.1	Verify that the user’s session inactivity timeout and absolute maximum session lifetime are documented, are appropriate in combination with other controls, and that the documentation includes justification for any deviations from NIST SP 800-63B re-authentication requirements.	2
7.1.2	Verify that the documentation defines how many concurrent (parallel) sessions are allowed for one account as well as the intended behaviors and actions to be taken when the maximum number of active sessions is reached.	2
7.1.3	Verify that all systems that create and manage user sessions as part of a federated identity management ecosystem (such as SSO systems) are documented along with controls to coordinate session lifetimes, termination, and any other conditions that require re-authentication.	2

V7.2 Fundamental Session Management Security

This section satisfies the essential requirements of secure sessions by verifying that session tokens are securely generated and validated.

#	Description	Level
7.2.1	Verify that the application performs all session token verification using a trusted, backend service.	1
7.2.2	Verify that the application uses either self-contained or reference tokens that are dynamically generated for session management, i.e. not using static API secrets and keys.	1
7.2.3	Verify that if reference tokens are used to represent user sessions, they are unique and generated using a cryptographically secure pseudo-random number generator (CSPRNG) and possess at least 128 bits of entropy.	1
7.2.4	Verify that the application generates a new session token on user authentication, including re-authentication, and terminates the current session token.	1

V7.3 Session Timeout

Session timeout mechanisms serve to minimize the window of opportunity for session hijacking and other forms of session abuse. Timeouts must satisfy documented security decisions.

#	Description	Level
7.3.1	Verify that there is an inactivity timeout such that re-authentication is enforced according to risk analysis and documented security decisions.	2
7.3.2	Verify that there is an absolute maximum session lifetime such that re-authentication is enforced according to risk analysis and documented security decisions.	2

V7.4 Session Termination

Session termination may be handled either by the application itself or by the SSO provider if the SSO provider is handling session management instead of the application. It may be necessary to decide whether the SSO provider is in scope when considering the requirements in this section as some may be controlled by the provider.

Session termination should result in requiring re-authentication and be effective across the application, federated login (if present), and any relying parties.

For stateful session mechanisms, termination typically involves invalidating the session on the back-end. In the case of self-contained tokens, additional measures are required to revoke or block these tokens, as they may otherwise remain valid until expiration.

#	Description	Level
7.4.1	Verify that when session termination is triggered (such as logout or expiration), the application disallows any further use of the session. For reference tokens or stateful sessions, this means invalidating the session data at the application backend. Applications using self-contained tokens will need a solution such as maintaining a list of terminated tokens, disallowing tokens produced before a per-user date and time or rotating a per-user signing key.	1
7.4.2	Verify that the application terminates all active sessions when a user account is disabled or deleted (such as an employee leaving the company).	1
7.4.3	Verify that the application gives the option to terminate all other active sessions after a successful change or removal of any authentication factor (including password change via reset or recovery and, if present, an MFA settings update).	2
7.4.4	Verify that all pages that require authentication have easy and visible access to logout functionality.	2
7.4.5	Verify that application administrators are able to terminate active sessions for an individual user or for all users.	2

V7.5 Defenses Against Session Abuse

This section provides requirements to mitigate the risk posed by active sessions that are either hijacked or abused through vectors that rely on the existence and capabilities of active user sessions. For example, using malicious content execution to force an authenticated victim browser to perform an action using the victim's session.

Note that the level-specific guidance in the “Authentication” chapter should be taken into account when considering requirements in this section.

#	Description	Level
7.5.1	Verify that the application requires full re-authentication before allowing modifications to sensitive account attributes which may affect authentication such as email address, phone number, MFA configuration, or other information used in account recovery.	2
7.5.2	Verify that users are able to view and (having authenticated again with at least one factor) terminate any or all currently active sessions.	2

#	Description	Level
7.5.3	Verify that the application requires further authentication with at least one factor or secondary verification before performing highly sensitive transactions or operations.	3

V7.6 Federated Re-authentication

This section relates to those writing Relying Party (RP) or Identity Provider (IdP) code. These requirements are derived from the NIST SP 800-63C for Federation & Assertions.

#	Description	Level
7.6.1	Verify that session lifetime and termination between Relying Parties (RPs) and Identity Providers (IdPs) behave as documented, requiring re-authentication as necessary such as when the maximum time between IdP authentication events is reached.	2
7.6.2	Verify that creation of a session requires either the user's consent or an explicit action, preventing the creation of new application sessions without user interaction.	2

References

For more information, see also:

- OWASP Web Security Testing Guide: Session Management Testing
- OWASP Session Management Cheat Sheet

V8 Authorization

Control Objective

Authorization ensures that access is granted only to permitted consumers (users, servers, and other clients). To enforce the Principle of Least Privilege (POLP), verified applications must meet the following high-level requirements:

- Document authorization rules, including decision-making factors and environmental contexts.
- Consumers should have access only to resources permitted by their defined entitlements.

V8.1 Authorization Documentation

Comprehensive authorization documentation is essential to ensure that security decisions are consistently applied, auditable, and aligned with organizational policies. This reduces the risk of unauthorized access by making security requirements clear and actionable for developers, administrators, and testers.

#	Description	Level
8.1.1	Verify that authorization documentation defines rules for restricting function-level and data-specific access based on consumer permissions and resource attributes.	1
8.1.2	Verify that authorization documentation defines rules for field-level access restrictions (both read and write) based on consumer permissions and resource attributes. Note that these rules might depend on other attribute values of the relevant data object, such as state or status.	2
8.1.3	Verify that the application's documentation defines the environmental and contextual attributes (including but not limited to, time of day, user location, IP address, or device) that are used in the application to make security decisions, including those pertaining to authentication and authorization.	3
8.1.4	Verify that authentication and authorization documentation defines how environmental and contextual factors are used in decision-making, in addition to function-level, data-specific, and field-level authorization. This should include the attributes evaluated, thresholds for risk, and actions taken (e.g., allow, challenge, deny, step-up authentication).	3

V8.2 General Authorization Design

Implementing granular authorization controls at the function, data, and field levels ensures that consumers can access only what has been explicitly granted to them.

#	Description	Level
8.2.1	Verify that the application ensures that function-level access is restricted to consumers with explicit permissions.	1
8.2.2	Verify that the application ensures that data-specific access is restricted to consumers with explicit permissions to specific data items to mitigate insecure direct object reference (IDOR) and broken object level authorization (BOLA).	1

#	Description	Level
8.2.3	Verify that the application ensures that field-level access is restricted to consumers with explicit permissions to specific fields to mitigate broken object property level authorization (BOPLA).	2
8.2.4	Verify that adaptive security controls based on a consumer's environmental and contextual attributes (such as time of day, location, IP address, or device) are implemented for authentication and authorization decisions, as defined in the application's documentation. These controls must be applied when the consumer tries to start a new session and also during an existing session.	3

V8.3 Operation Level Authorization

The immediate application of authorization changes in the appropriate tier of an application's architecture is crucial to preventing unauthorized actions, especially in dynamic environments.

#	Description	Level
8.3.1	Verify that the application enforces authorization rules at a trusted service layer and doesn't rely on controls that an untrusted consumer could manipulate, such as client-side JavaScript.	1
8.3.2	Verify that changes to values on which authorization decisions are made are applied immediately. Where changes cannot be applied immediately, (such as when relying on data in self-contained tokens), there must be mitigating controls to alert when a consumer performs an action when they are no longer authorized to do so and revert the change. Note that this alternative would not mitigate information leakage.	3
8.3.3	Verify that access to an object is based on the originating subject's (e.g. consumer's) permissions, not on the permissions of any intermediary or service acting on their behalf. For example, if a consumer calls a web service using a self-contained token for authentication, and the service then requests data from a different service, the second service will use the consumer's token, rather than a machine-to-machine token from the first service, to make permission decisions.	3

V8.4 Other Authorization Considerations

Additional considerations for authorization, particularly for administrative interfaces and multi-tenant environments, help prevent unauthorized access.

#	Description	Level
8.4.1	Verify that multi-tenant applications use cross-tenant controls to ensure consumer operations will never affect tenants with which they do not have permissions to interact.	2
8.4.2	Verify that access to administrative interfaces incorporates multiple layers of security, including continuous consumer identity verification, device security posture assessment, and contextual risk analysis, ensuring that network location or trusted endpoints are not the sole factors for authorization even though they may reduce the likelihood of unauthorized access.	3

References

For more information, see also:

- OWASP Web Security Testing Guide: Authorization
- OWASP Authorization Cheat Sheet

V9 Self-contained Tokens

Control Objective

The concept of a self-contained token is mentioned in the original RFC 6749 OAuth 2.0 from 2012. It refers to a token containing data or claims on which a receiving service will rely to make security decisions. This should be differentiated from a simple token containing only an identifier, which a receiving service uses to look up data locally. The most common examples of self-contained tokens are JSON Web Tokens (JWTs) and SAML assertions.

The use of self-contained tokens has become very widespread, even outside of OAuth and OIDC. At the same time, the security of this mechanism relies on the ability to validate the integrity of the token and to ensure that the token is valid for a particular context. There are many pitfalls with this process, and this chapter provides specific details of the mechanisms that applications should have in place to prevent them.

V9.1 Token source and integrity

This section includes requirements to ensure that the token has been produced by a trusted party and has not been tampered with.